

Adversarial Search

Game Playing

Minimax search

Alpha-Beta pruning

Dr. Manas Ranjan Patra
NIST University

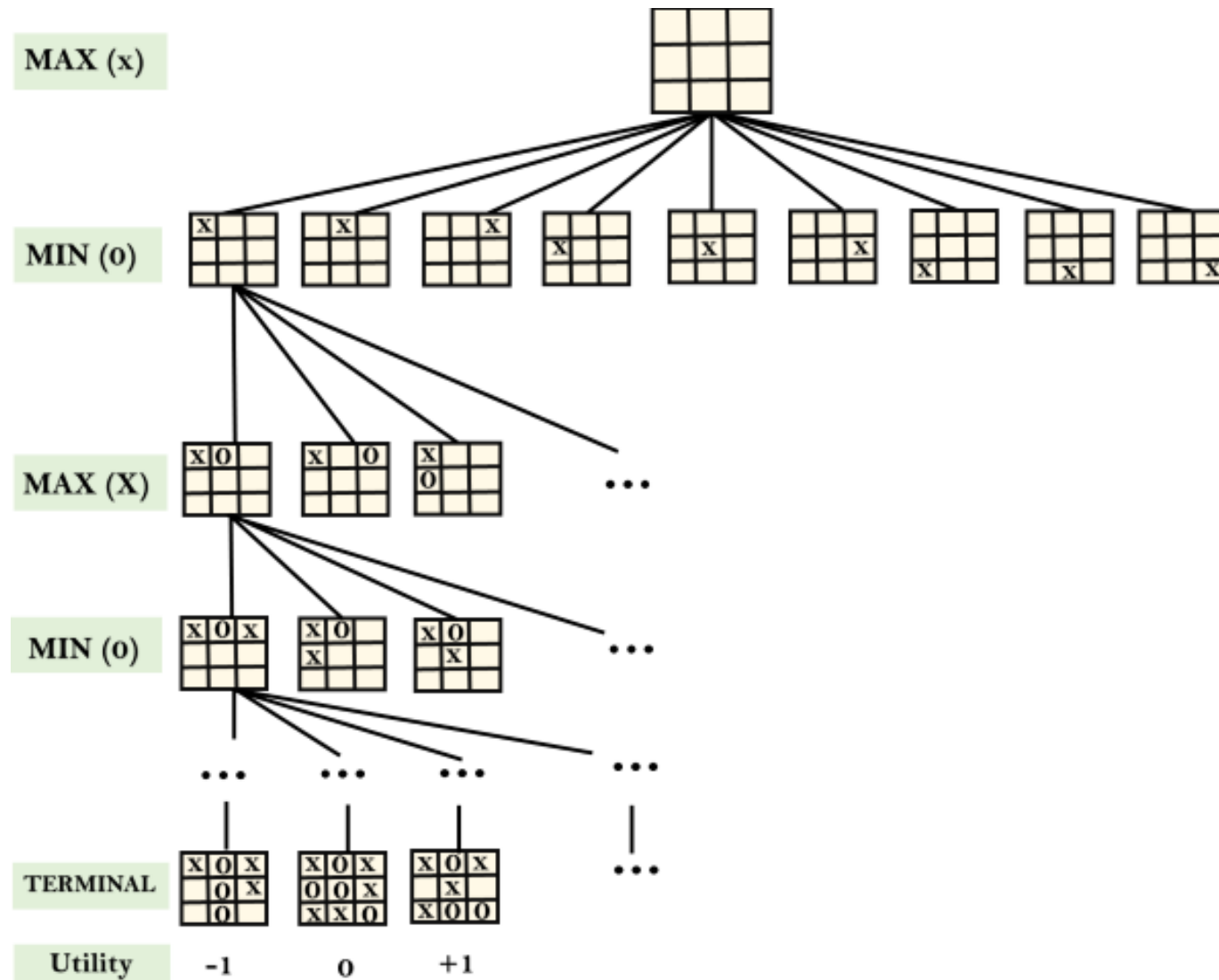
Adversarial search (Game)

- So far we have studied search strategies which are associated with a single agent.
- Searches in which two or more players with conflicting goals try to explore the same search space for a solution, those are referred to as **adversarial searches**, often called as **Games**.
- Adversarial search is used in decision-making for competitive environments, where two agents (players) have opposing goals. In such environments, one player tries to maximize their performance, while the other tries to minimize it. The **minimax** algorithm is a common approach to solve adversarial search problems.
- Games are modeled as Search problems and heuristic evaluation functions.
- Each move of a player in the game is called as **ply**.

Components of Adversarial Search

- 1. Two players:** One is called the "Maximizer" (MAX), who tries to maximize the score, and the other is called the "Minimizer" (MIN), who tries to minimize the score.
- 2. Game tree:** The game can be represented as a tree, where:
 1. Nodes represent states of the game.
 2. Edges represent actions that transition between states.
 3. Leaf nodes represent terminal states (end of the game) with assigned utility values (e.g., win, lose, or draw).
- 3. Utility function:** Each terminal node has a utility value based on the outcome of the game (e.g., +1 for a win, 0 for a draw, and -1 for a loss).

Tic-Tac-Toe game tree



Explanation:

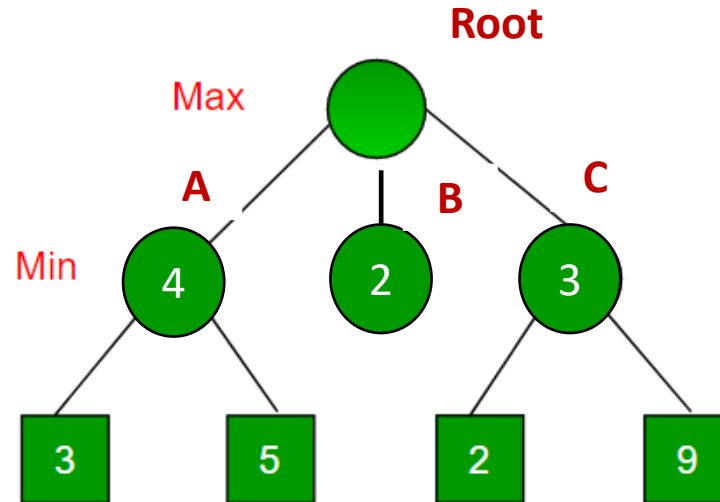
- From the initial state, MAX has 9 possible moves as he starts first. MAX places x and MIN places o, and both the players play alternatively until a leaf node is reached where one player has three (x or O) in a row/column/diagonal or when all the squares are filled.
- Each player does his best to prevent another one from winning.
- So in the game tree, we have a layer of Max, a layer of MIN, and each layer is called as **Ply**.

Mini-Max Algorithm

- Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that the opponent also plays optimally.
- In Minimax the two players are called **maximizer** and **minimizer**. The **maximizer** tries to get the highest score possible while the **minimizer** tries to do the opposite and get the lowest score possible.
- Every board state has a value associated with it. In a given state if the maximizer has upper hand then, the score of the board will tend to be some positive value. If the minimizer has the upper hand in that board state then it will tend to be some negative value. The values of the board are calculated by some heuristics which are unique for every type of game.

Example (two-ply game tree):

- Consider a game which has 4 final states and paths to reach final state are from root to 4 leaves. Assume that you have the 1st chance to play (maximizer) i.e., you are at the root.



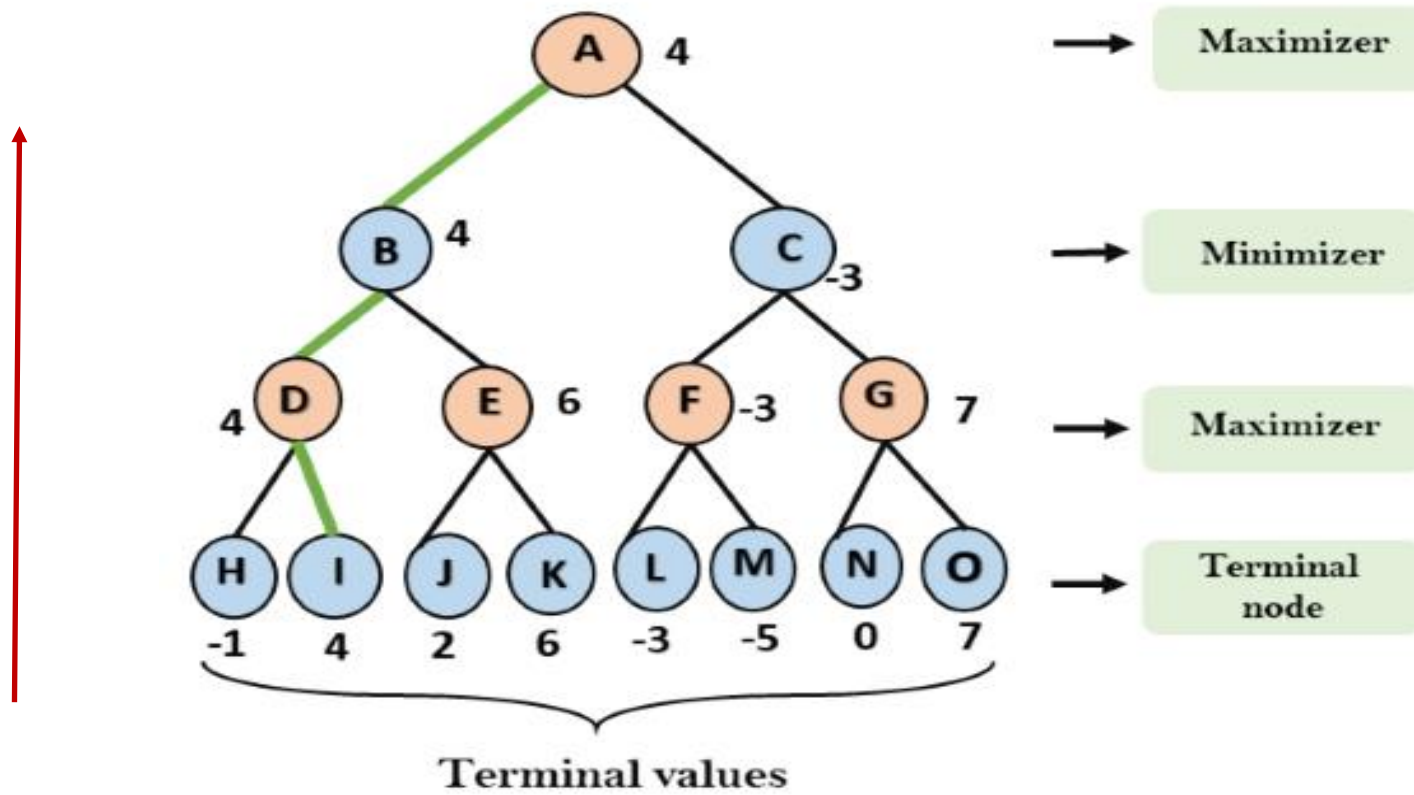
- It is a backtracking based algorithm. Therefore, it tries all possible moves, then backtracks and makes a decision.
- When the Maximizer goes to A (HF=4): It is now the minimizer's turn. The minimizer will choose 3.
- In case Maximizer goes to C (HF=3): The minimizer will choose 2.
- But, the Maximizer will certainly go to 4 (being a maximizer)**

Minimax Algorithm

```
1.  minimax(node, depth, is_maximizing_player)
2.    if node is a terminal node or depth == 0
3.      return utility(node)
4.
5.    if is_maximizing_player // for Maximizer Player
6.      max_eval =  $-\infty$  // indicates a very low starting value for the maximizer
7.      for each child of node
8.        eval = minimax(child, depth - 1, False)
9.        max_eval = max(max_eval, eval) //gives Maximum of the values
10.     return max_eval
11.  else // for Minimizer player
12.    min_eval =  $\infty$  //indicates a very high starting value for the minimizer
13.    for each child of node:
14.      eval = minimax(child, depth - 1, True)
15.      min_eval = min(min_eval, eval) //gives minimum of the values
16.  return min_eval
```

First the algorithm generates the entire game-tree and apply the utility function to get the utility values for nodes.

Next, the nodes are evaluated from the point of view of the maximizer & Minimizer to select the nodes at each step (by backtracking whenever necessary to find the best move).



Drawback

- Complex games (like Chess) have huge number of alternatives at every step (large branching factor), and a player has lots of choices to decide. This slows down the searching process.
- This limitation of the minimax algorithm can be improved using **alpha-beta pruning**.

Alpha-Beta Pruning

- It reduces the number of nodes that need to be evaluated by "pruning" (cutting off) branches that would not affect the final decision.
- It is an optimization technique for the minimax algorithm.
- As the depth of the tree increases the alternative states also increase exponentially.
- Alpha-beta pruning is a technique to eliminate some of the alternatives.
- This involves two threshold parameter **Alpha** and **beta**
- Alpha-beta pruning can be applied at any depth of a game tree.
- It can prune both the leaf nodes as well as an entire branch.

Threshold parameter

- **Alpha:** The best value (highest value) that the maximizer (MAX) can guarantee at any point. It starts at $-\infty$ and gets updated as the algorithm explores options.
- **Beta:** The best value (lowest-value) that the minimizer (MIN) can guarantee at any point. It starts at $+\infty$ and decreases as the algorithm proceeds.
- Pruning can make the search process faster.

Process

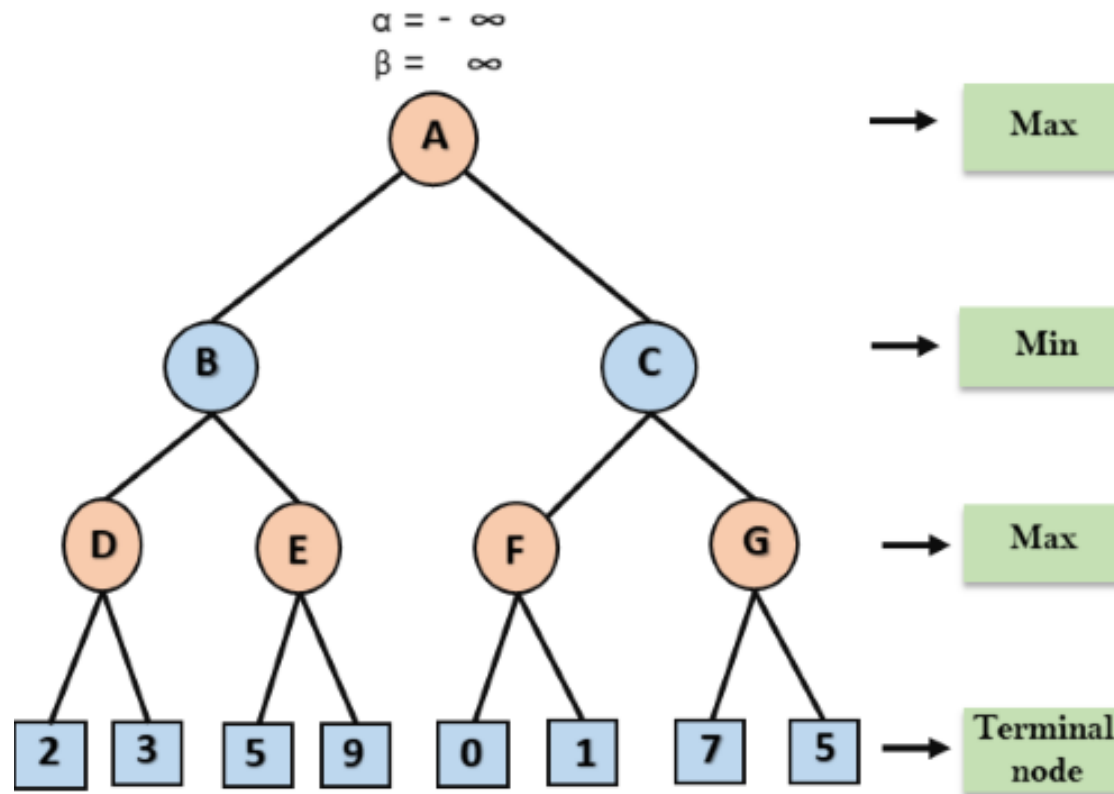
- The Max player only updates the value of alpha.
- The Min player only updates the value of beta.
- While backtracking the tree, the node values are passed to upper nodes (and not the values of alpha & beta).
- The values of alpha & beta are only passed to the child nodes.
- Conditions for Pruning:

Beta cutoff ($\beta \leq \alpha$): This means that the minimizer has found a better option than what the maximizer is hoping for, so the rest of the branches under this node won't be evaluated.

Alpha-Beta Pruning steps

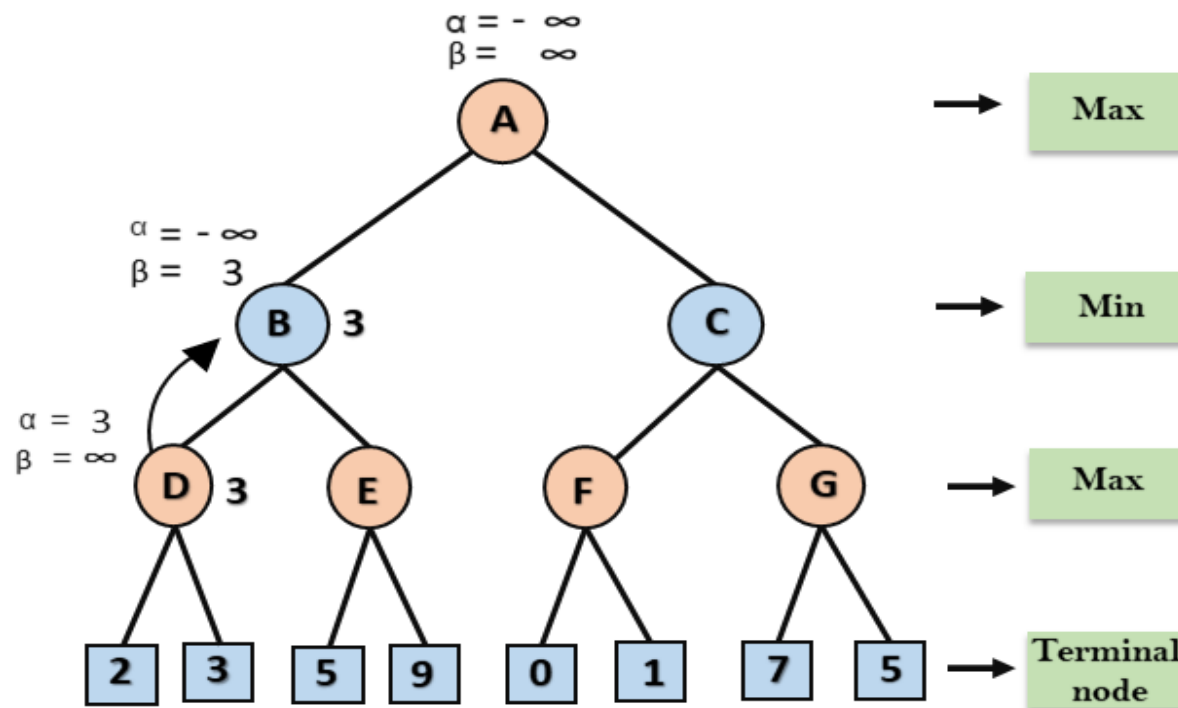
- This is similar to the minimax algorithm, but two variables, α and β are introduced to keep track of the best choices available for MAX and MIN.
- **Maximizer's turn (MAX):** Tries to increase α
- **Minimizer's turn (MIN):** Tries to reduce β
- While traversing the game tree, **Prune** the branches that won't improve the outcome for either player. If the value of a node is worse than what a previous sibling node had already guaranteed, then that branch is "pruned" because it can't possibly affect the outcome.

Step 1: At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these values of alpha and beta are passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



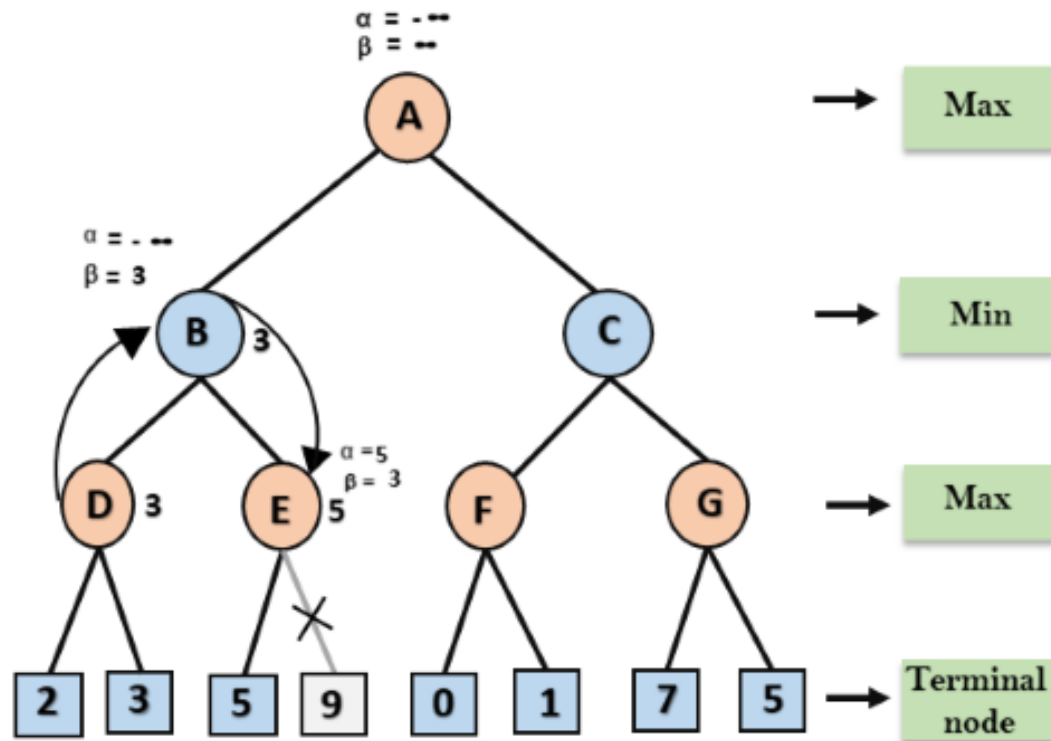
Step 2: At Node D, the value of α will be calculated as it is the turn for Max. The value of α is compared with 2 and then 3, and the max ($2, 3$) = 3 will be the value of α at node D and node value will also be 3.

Step 3: Now algorithm backtracks to node B, where the value of β will change as this is the turn of Min, Now $\beta = +\infty$, will be compared with the available subsequent nodes value, i.e. $\min(\infty, 3) = 3$, hence at node B now $\alpha = -\infty$, and $\beta = 3$.



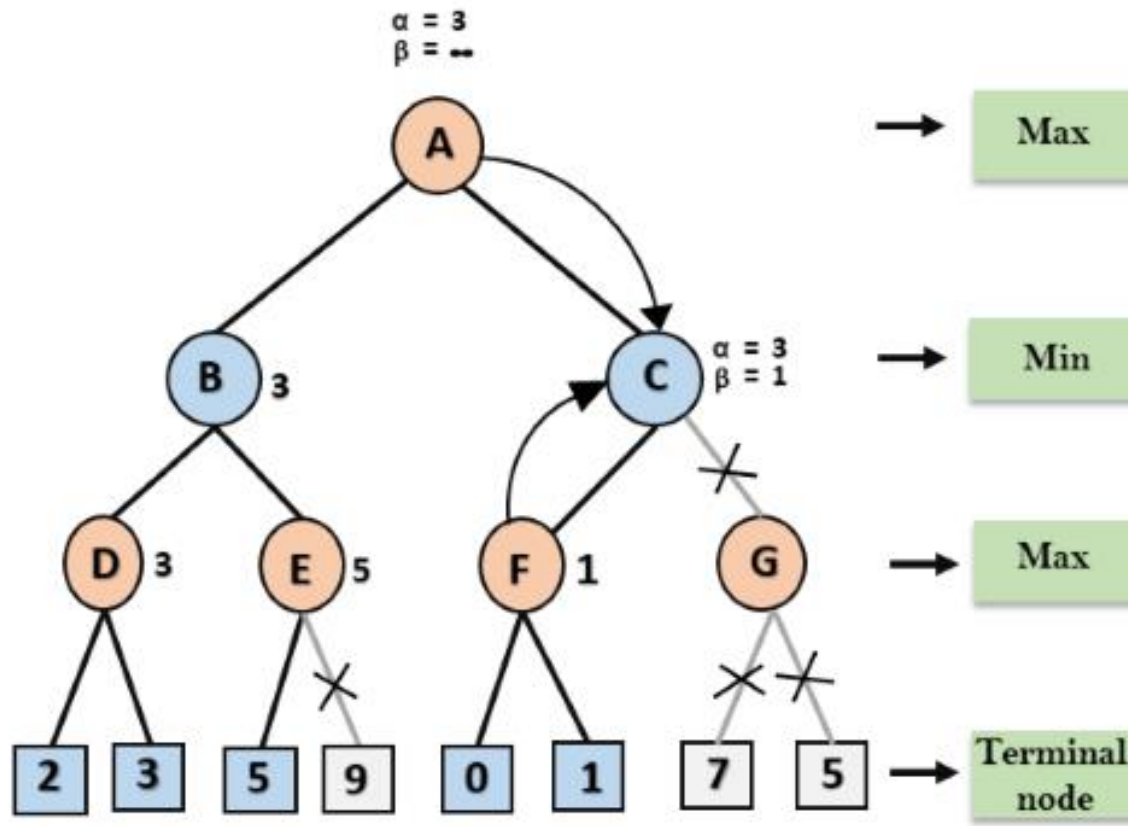
In the next step, algorithm traverses the next successor of Node B which is **node E**, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.

Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E, $\alpha = 5$ and $\beta = 3$. Now, $\alpha \geq \beta$, so the right successor of E will be pruned, and it will not be traversed, and the value at node E will be 5.

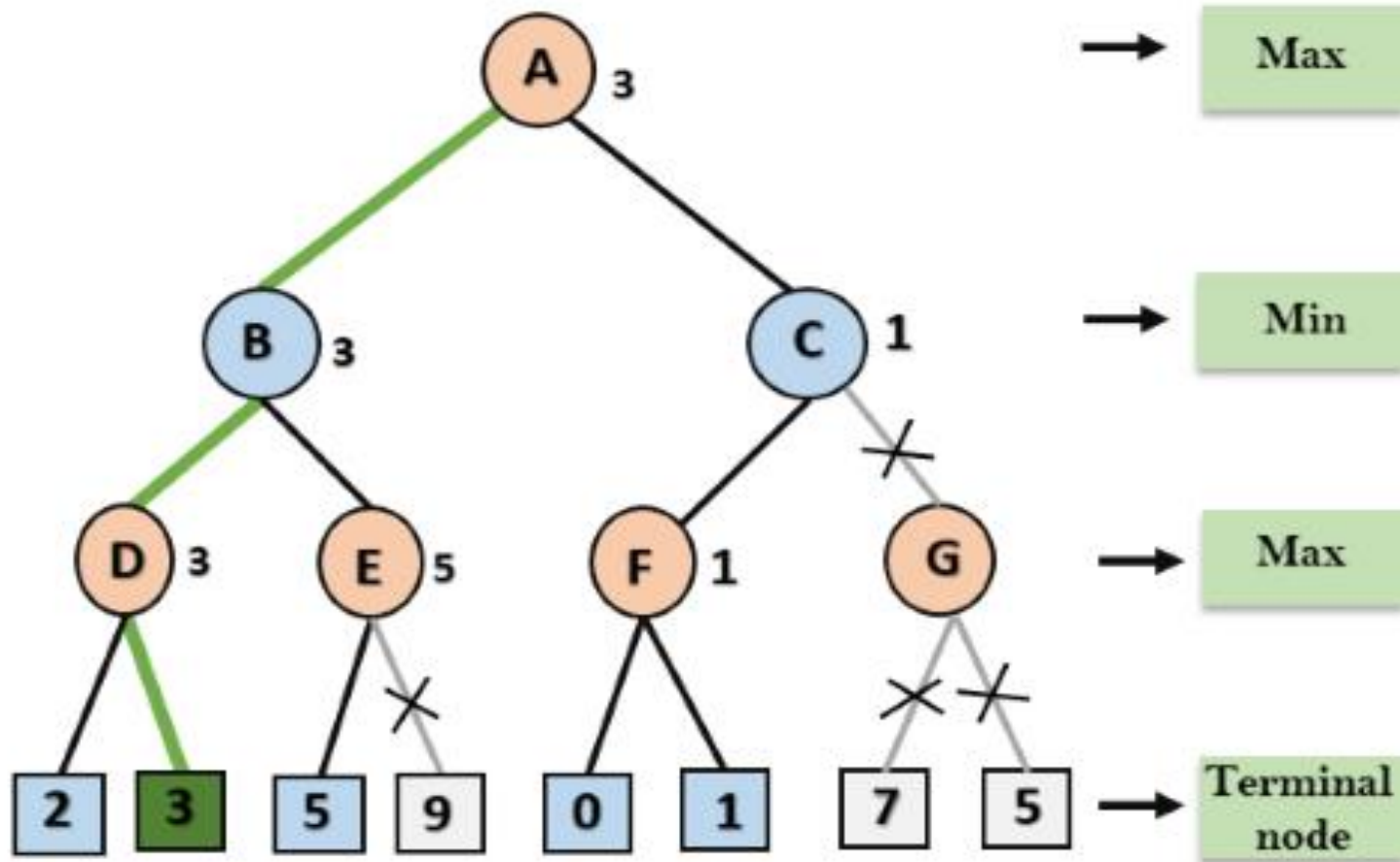


At node C, $\alpha=3$ and $\beta=+\infty$, and the same values will be passed on to node F.

Step 7: Node F returns the node value 1 to node C, at C $\alpha = 3$ and $\beta = +\infty$, here the value of beta will be changed to 1 as $\min(\infty, 1) = 1$. Now at C, $\alpha = 3$ and $\beta = 1$, and again it satisfies the condition $\alpha \geq \beta$, so the next child of C which is G will be pruned, and the algorithm will not compute that branch.



Step 8: C now returns the value 1 to A. Here the best value for A is $\max(3, 1) = 3$. The final game tree is shown below. **Thus, the optimal value for the maximizer is 3.**



Move Ordering in Alpha-Beta pruning

The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined.

- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^d)$ where b is the branching factor, and d is the depth of the tree.
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first searches left of the tree and goes deep into the tree. Complexity in ideal ordering is $O(b^{d/2})$.

Algorithm minimax(node, depth, alpha, beta, maximizingPlayer)

1. **if** depth ==0 or node is a terminal node then
2. **return static** evaluation of node
3. **if** MaximizingPlayer then // for Maximizer Player
4. maxEva= -infinity
5. **for** each child of node **do**
6. eva= minimax(child, depth-1, alpha, beta, False)
7. maxEva= max(maxEva, eva)
8. alpha= max(alpha, maxEva)
9. **if** alpha >= beta
10. **break**
11. **return** maxEva
12. **else** // for Minimizer player
13. minEva= +infinity
14. **for** each child of node **do**
15. eva= minimax(child, depth-1, alpha, beta, **true**)
16. minEva= min(minEva, eva)
17. beta= min(beta, eva)
18. **if** alpha >= beta
19. **break**
20. **return** minEva

Benefits of Alpha-Beta Pruning:

- **Efficiency:** By eliminating unnecessary branches in the search tree, it reduces the number of nodes to be evaluated. Thus, reduces computational efforts.
- **Optimal result:** The final result remains the same as the regular minimax algorithm, but the process is faster.